

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2023

M. Frisbie, *Building Browser Extensions*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5_15

15. Cross-Browser Extensions

Matt Frisbie¹

(1) California, CA, USA

When it comes to supporting multiple platforms, browser extensions exist somewhere in between websites and mobile apps. One end of the spectrum is the website, which enjoys a “write-once-run-anywhere” developer experience. On the other end of the spectrum is the mobile app, which is written in a proprietary language and will only work on a specific operating system and a subset of device types. In this chapter, we will discuss what challenges and tradeoffs are involved when supporting an extension on multiple browsers.

Introduction to Cross-Browser Support

Website code enjoys a consistent host environment. You can write a website once, load it on any OS or device or browser, and expect that it will work correctly. There are some corner cases where browsers diverge, but these are relatively rare. All browser vendors spend lots of time ensuring that their product conforms to the W3C standards and will render a web page more or less in the same way as any other browser.

Mobile apps run in a proprietary host environment. If you want to have a mobile app run on multiple operating systems, code written for Android is nearly useless when it comes to building for iOS. Apps are distributed in different places, written in different languages, built for different devices, and utilize completely different SDKs.

Browser extensions borrow a little bit from each of these modalities. Extensions use an API that is standardized across browsers, but not all browsers support the full API. Extensions all are built with HTML, JavaScript, and CSS, but the manifest support between browsers has certain idiosyncrasies and incompatibilities. Extensions are distributed in browser-specific app stores, but a majority of web browsers are built atop Chromium – meaning they all can install apps distributed by the Chrome Web Store.

Browser Coverage Tradeoffs

When choosing if and to what extent you will support multiple browsers, you should begin by asking yourself who your target user is. For a majority of extensions, the answer to this question will be, “as many people as possible”! Supporting all major browsers is certainly possible, but depending on your extension, this may be overkill. Variable API support,

multiple app stores, and idiomatic JS/CSS engines can ensnare a developer's time.

Browser Share

Fortunately, for extensions that are just getting started, you can very easily support a majority of users with a single codebase. Consider the following metrics for desktop browser share in 2022 (Figure 15-1).

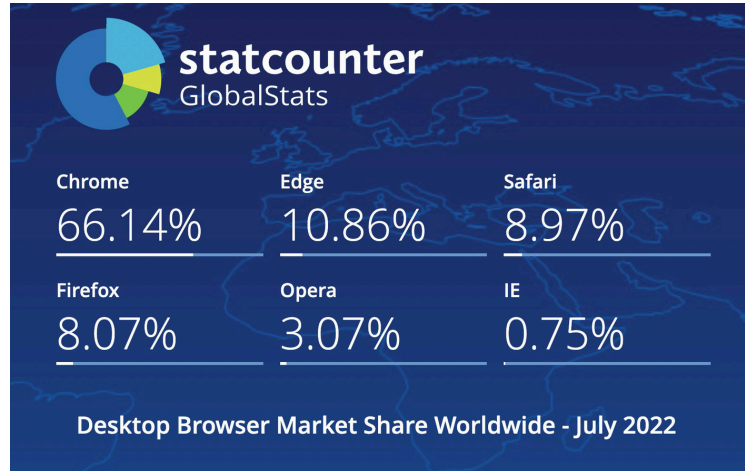


Figure 15-1 Desktop browser market share in 2022

Google Chrome dominates the market at 66%. If you were to deploy your extension to just the Chrome Web Store, you are automatically covering 2/3 of the market.

Chromium Browser Extension Sharing

Fortunately, the modern browser landscape is working to your advantage. Many major browsers such as Edge, Opera, Vivaldi, and Brave are all build upon the open-source Chromium browser. As a result, all these browsers feature nearly perfect compatibility with extensions built for Google Chrome. Each of these browsers can either directly install from, or be configured to install from, the Chrome Web Store. Therefore, an extension published only to the Chrome Web Store can in fact be installed by Chrome, Edge, and Opera: **over 80%** of desktop traffic!

Note There are minor differences between Google Chrome and other Chromium browsers, such as the identity API. Nevertheless, the overall API parity between all Chromium browsers is nearly seamless.

Adapting Your Codebase

Depending on how broad you want your extension support to be, organizing your codebase may require some finesse. In this section, we'll cover a few simple principles that can be used.

API Probing

Not all browsers support the full WebExtensions API. If a particular method is critical to your extension, and a browser does not support it,

then there is little you can do. In situations like these, it is probably best to *not* support the browser, and encourage the user to use a supported browser.

If, however, the API you wish to use is not critical, and the extension can function without it (even in a degraded state), then you can safely probe for an API or a method to check if it is available. For example:

```
if (chrome.storage.session) {  
  // API is available, proceed normally  
} else {  
  // API is not available. UX options:  
  // - Fall back to similar API  
  // - Disable or hide buttons  
  // - Notify user  
  // - Silently degrade  
}
```

Differential Manifests

When working in development mode, you will find that most browsers are relatively permissive when it comes to manifest structure. However, when deploying to a marketplace, you may run into a situation where there are strict incompatibilities between what two different marketplaces allow or disallow.

To address this, you will need to generate differential manifests for each deployment bundle. A common pattern is to maintain a single “master” manifest, and then perform ad-hoc modifications specific to each marketplace when generating the deployment bundles.

Tip Differential manifest generation can be quite tedious. Extension development platforms such as Plasmio (<https://www.plasmio.com/>) can automate this process for you.

Manifest v2/v3

At the time this book was published, extension developers are stuck in an unenviable position where some marketplaces like the Chrome Web Store are only accepting manifest v3, and others like the Firefox Addon marketplace are only accepting manifest v2. It seems very likely that this status quo is transitory, and soon all major marketplaces will coalesce around manifest v3. I would not recommend splitting support between manifest v2 and v3, the chasm between how the two behave is not worth bridging; you should target support for manifest v3.

Extension Marketplaces

Just as mobile apps must be installed from an app store, extensions must be installed from an extension marketplace. All major browsers maintain their own extension store.

Marketplace Similarities

The following is a list of features that are consistent between extension marketplaces:

- **A developer account is required.** No marketplace allows you to publish a browser extension anonymously. You can always create a separate developer account that does not identify you personally.
- **All new extensions and extension updates undergo review.** Every extension version submitted will undergo some kind of review process before it is approved. Some reviews will be automated, and some will be manual. The permissions you request in your extension manifest may change how closely your extension is scrutinized.
- **All extensions require some copy and assets for the listing.** Icons, titles, descriptions, promotional images, and links to host websites are just a few of the things you can provide when publishing an extension. Some marketplaces will automatically draw a portion of this content from the manifest automatically. Additionally, different marketplaces will require a different content.
- **All marketplaces support manual publishing (except Safari).** This involves uploading a zip file containing all the assets and files for the extension. The store will perform an automated analysis of this zip file to check if it is valid, and if it is, add it to the review queue.
- **All marketplaces support automated publishing.** This is covered in the *Extension Development and Deployment* chapter.

Marketplace Differences

The following is a list of features that are different between extension marketplaces:

- **Marketplaces have different automated file checking processes.** Linting, filetype checks, file sizes, and manifest validation are just a few of the categories that each marketplace will implement slightly differently.
- **Marketplaces will review submissions at different speeds.** The Chrome Web Store typically takes a few days to approve submissions, whereas the Mozilla Add-ons marketplace approves submissions almost instantly.

Chrome Web Store

<https://chrome.google.com/webstore>

The Chrome Web Store is by far the largest and most popular marketplace for browser extensions (Figure 15-2). It has 180,000 Google Chrome extensions available, which is roughly 10% of the Apple App Store's 1.8 million mobile apps. As mentioned earlier in the chapter, all Chromium browsers are capable of installing extensions directly from this marketplace. To publish an extension, you will need a Chrome Web Store developer account. It requires a one-time \$5 registration fee.

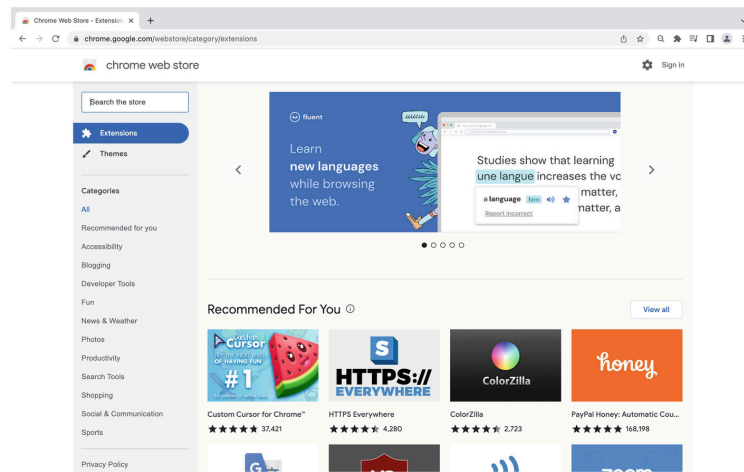


Figure 15-2 The Chrome Web Store web page

Add-ons for Firefox

<https://addons.mozilla.org/>

It is free to publish extensions to the Firefox extensions marketplace (Figure 15-3).

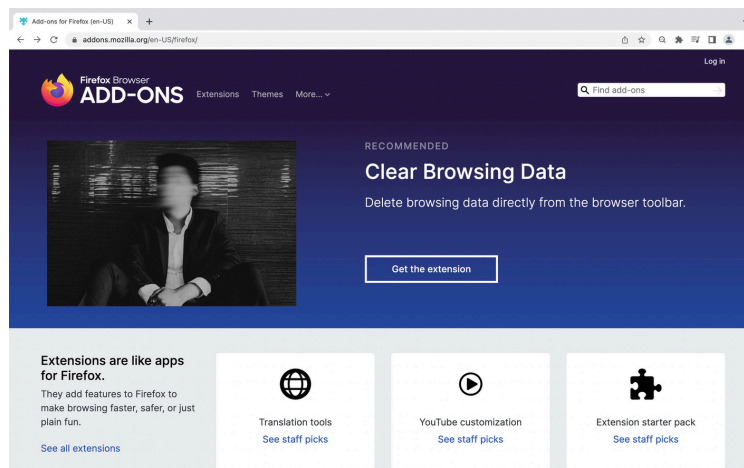


Figure 15-3 The Firefox Add-ons web page

Microsoft Edge Add-ons

<https://microsoftedge.microsoft.com/addons/Microsoft-Edge-Extensions-Home>

It is free to publish extensions to the Microsoft Edge extensions marketplace (Figure 15-4).

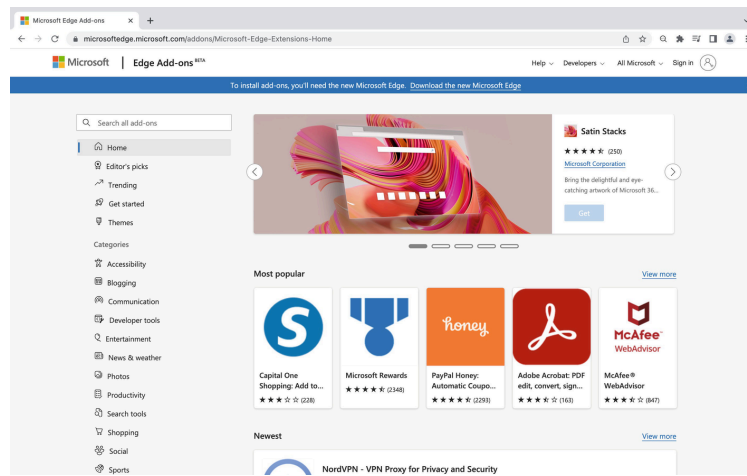


Figure 15-4 The Microsoft Edge Add-ons web page

Safari Extensions App Store

The Safari Extension Store is only accessible through the Safari browser (Figure 15-5). Publishing an extension to the App Store requires an Apple Developer Account, which is \$99/year. You will also need Xcode installed to build the extension and deploy it to the app store.

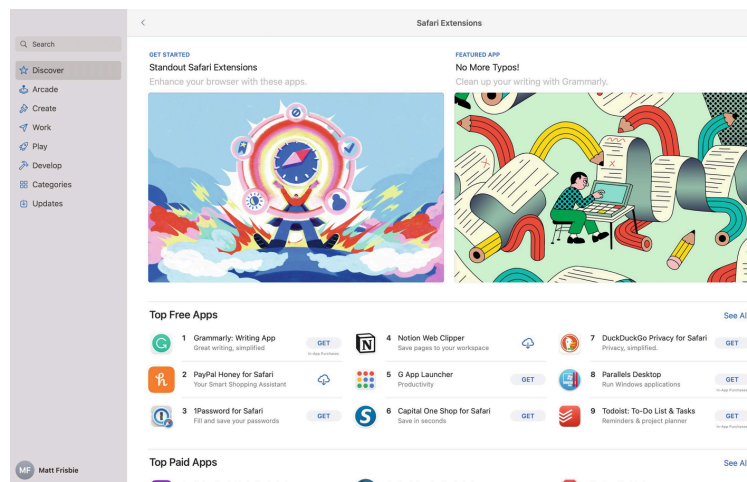


Figure 15-5 The Apple Safari Extensions App Store

Opera Addons

<https://addons.opera.com/>

It is free to publish extensions to the Opera extensions marketplace (Figure 15-6).

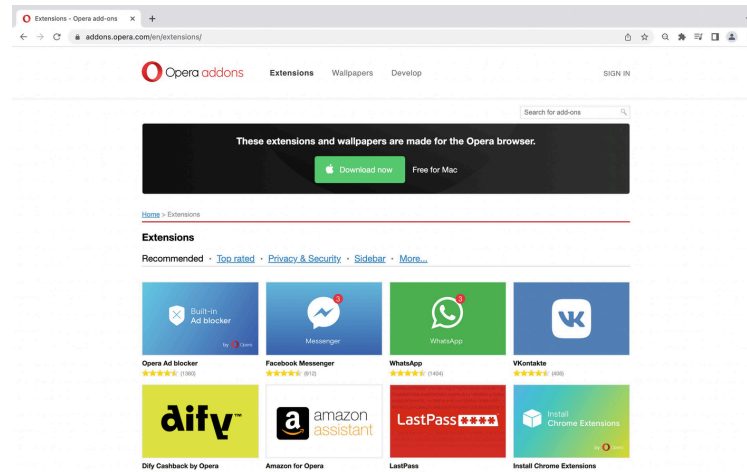


Figure 15-6 The Opera Add-ons web page

Mobile Extensions

Support for extensions is limited on mobile devices. Major browsers like Google Chrome do not support it on any OS, and Apple forbids non-Safari browsers from installing third-party extensions on iOS. However, on both Android and iOS devices it is still possible to install browser extensions if you are using the appropriate browser. API support is mixed on mobile devices.

Mobile Extension User Interfaces

With respect to WebExtensions API support, mobile browsers that allow extensions generally have parity with their desktop equivalents. Popups, options pages, content scripts, and background pages all will still run and render. However, due to the mobile form factor, there are some UX differences to consider:

- The toolbar action button used to open the popup are accessible through the browser's menu button.
- The options page is generally accessed through a "Settings" button in the extension's management view. This is fairly hard to find for the user, so do not expect this to be a common entrypoint.
- Safari extensions are packaged as apps. Therefore, they include an additional native app user interface. This is discussed later in the chapter.
- On mobile devices, the popup page will typically render either as a standalone web page (Firefox, Kiwi) or as a native modal (Safari).
- The options page will always render as stand-alone web page.

Kiwi Browser

If you're looking to install Chrome extensions on an Android device, the best solution is to use the Kiwi Browser (Figure 15-7). The browser is based on Webkit and Chromium, and you can directly install extensions from the Chrome Web Store. Using extensions in the Kiwi browser is only possible on Android.

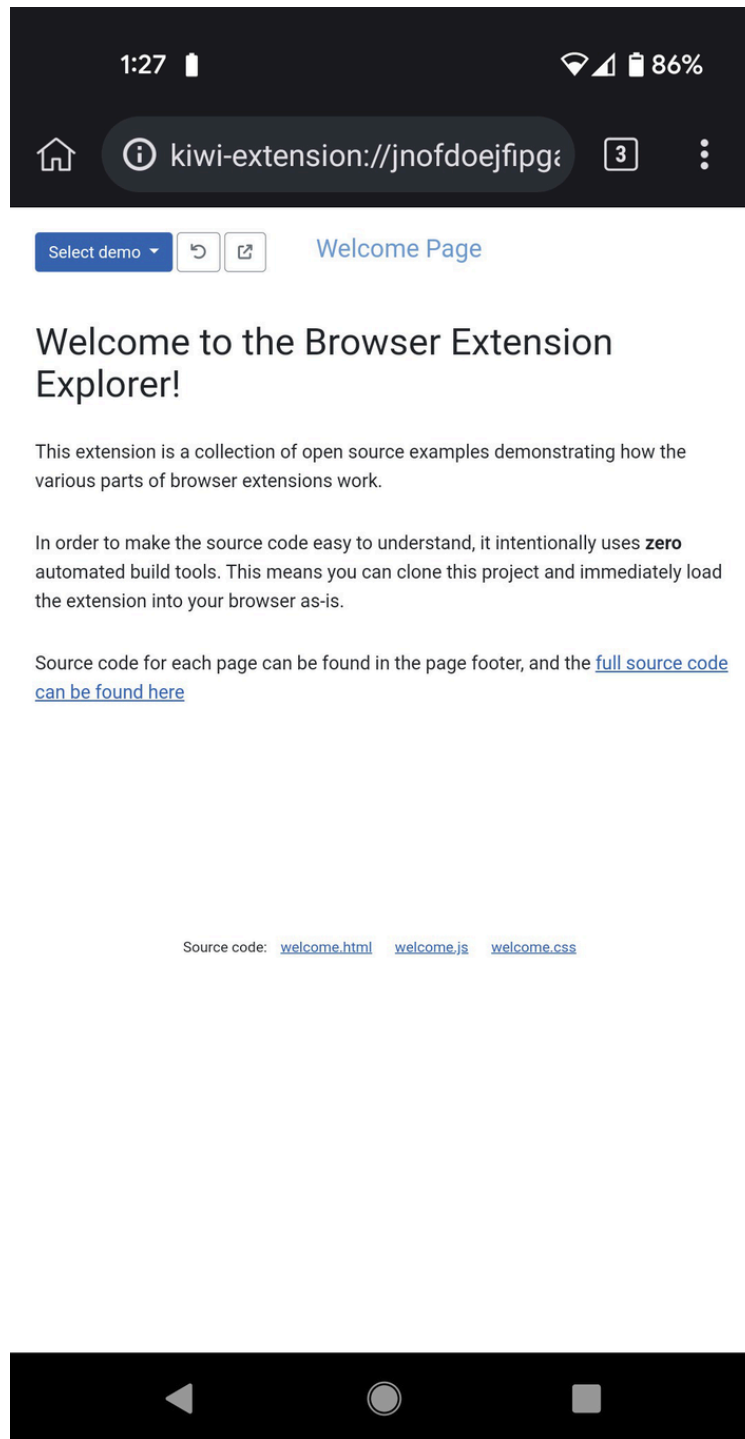


Figure 15-7 The Kiwi Browser showing an extension page

Firefox Mobile

Extensions published on the Firefox marketplace can choose to be installable for Firefox mobile (Figures 15-8 and 15-9). You can directly install extensions from the Firefox Add-ons marketplace. Extensions on the Firefox Mobile browser are only possible on Android.

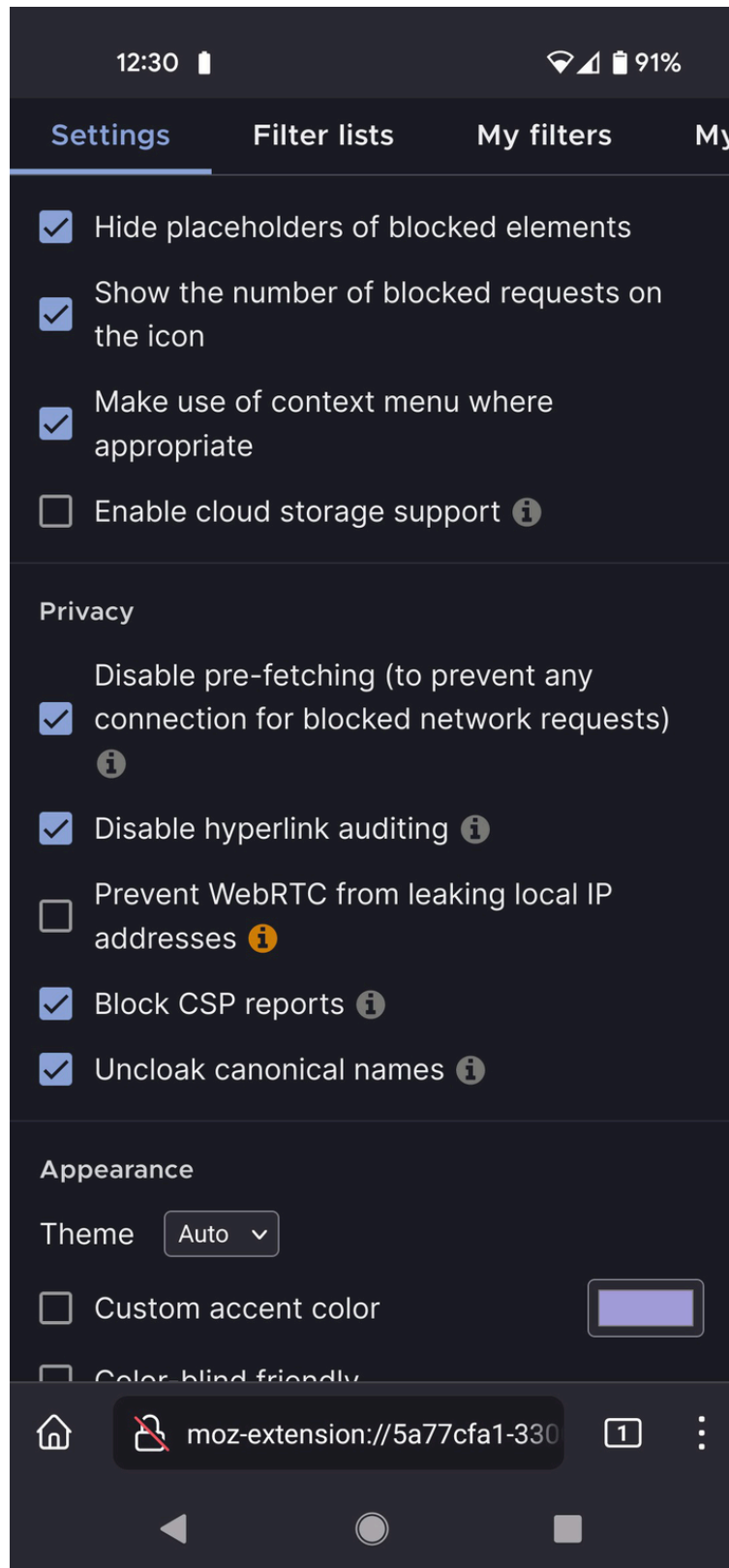


Figure 15-8 The uBlock Origin options page in Firefox Mobile

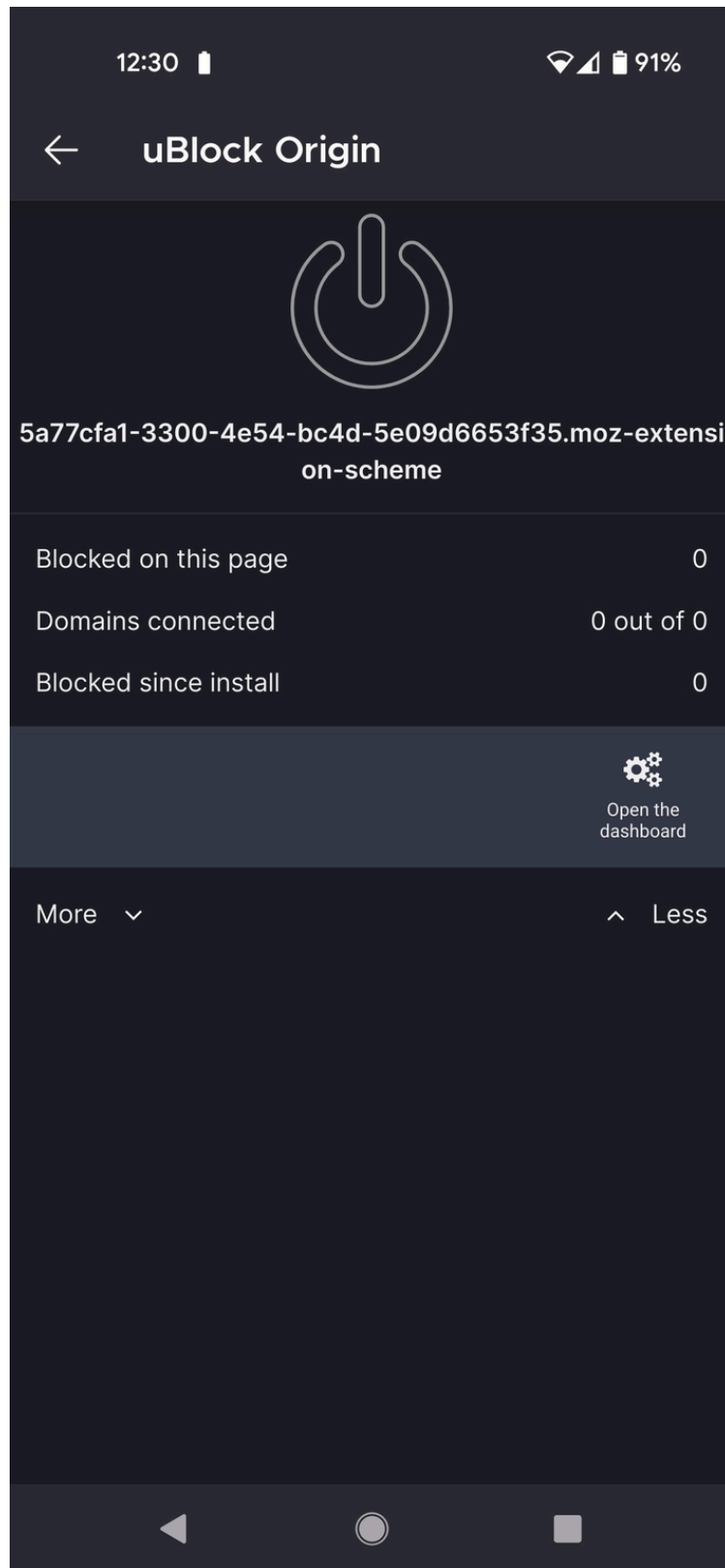


Figure 15-9 The uBlock Origin popup page in Firefox Mobile

iOS Safari

Safari extensions can be installed on iOS devices through the App Store. These are structured differently than other mobile extensions. Publishing a Safari extension for iOS involves a considerable amount of extra effort,

but at its core it still includes the same web architecture as other desktop extensions (Figures 15-10 and 15-11).

NoteSafari app development is covered in depth later in the chapter.

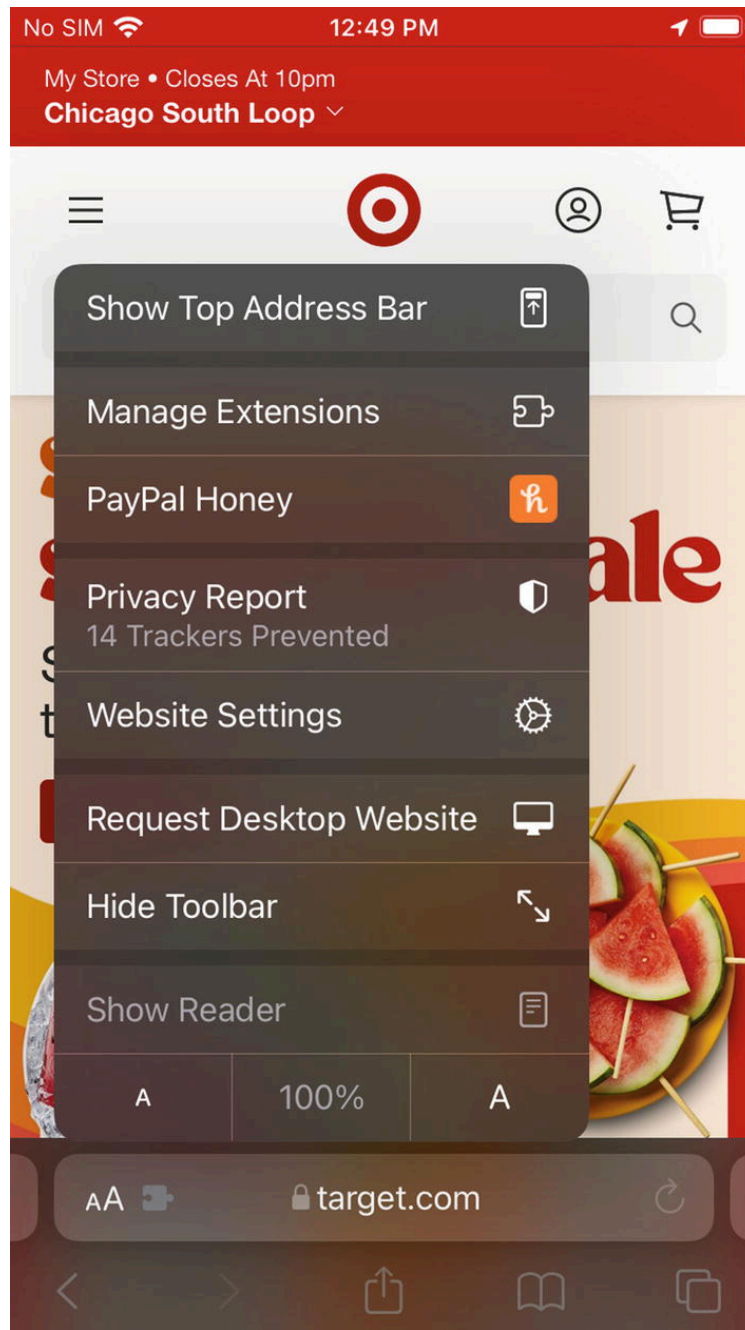
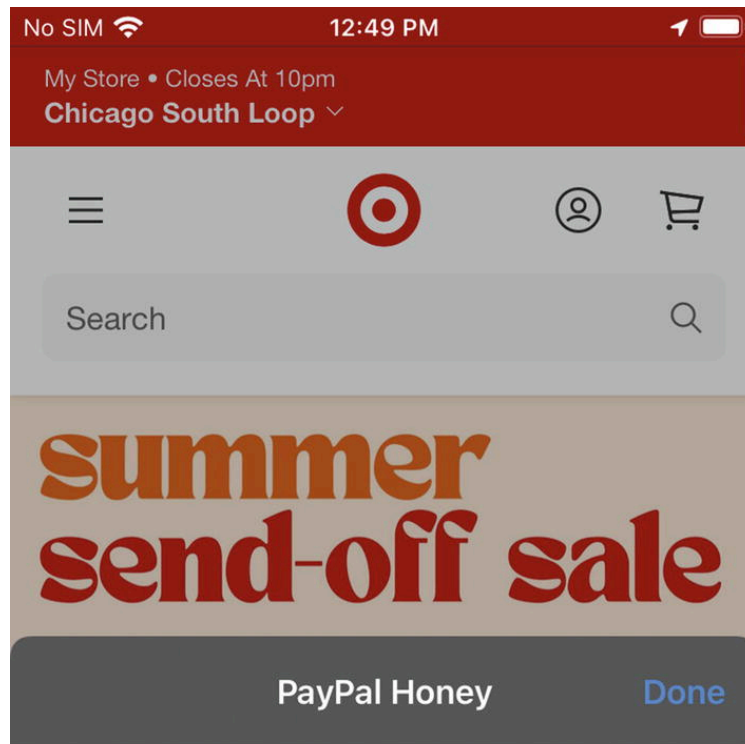


Figure 15-10 The Safari browser menu showing the options for an installed Extension



15 coupons found!

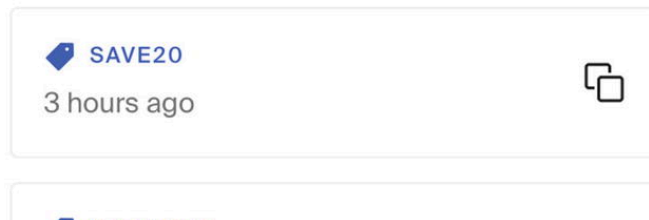


Figure 15-11 An extension popup page showing in the iOS Safari browser

Automated Deployment

As of 2022, all major extension marketplaces support automated deployment. The details of deploying to each store are out of the scope of this book, but you can find links to the documentation for each marketplace:

- **Chrome Web Store:**
https://developer.chrome.com/docs/webstore/api_index/
- **Microsoft Edge Add-ons Marketplace:**
<https://docs.microsoft.com/en-us/microsoft-edge/extensions-chromium/publish/api/using-addons-api>
- **Firefox Add-ons Marketplace:** <https://addons-server.readthedocs.io/en/latest/topics/api/signing.html>
- **Apple App Store:**
<https://developer.apple.com/documentation/appstoreconnectapi>

Tip Some extension development platforms such as Plasmo (<https://www.plasmo.com/>) can automate deployment of your extension to these marketplaces.

WebExtensions API Support

Not all browsers fully support the WebExtensions API, and some have support for non-standardized API features. These are changing all the time, so this book will only link to the following pages that catalog each browser's API support:

- **MDN table showing support across all browsers:**
https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Browser_support_for_JavaScript_APIs
- **Google Chrome API reference:**
<https://developer.chrome.com/docs/extensions/reference/>
- **Microsoft Edge API support:** <https://docs.microsoft.com/en-us/microsoft-edge/extensions-chromium/developer-guide/api-support>
- **Safari WebExtensions API compatibility:**
https://developer.apple.com/documentation/safariservices/safari_web_extensions/assessing_your_
- **Opera API support:** <https://dev.opera.com/extensions/apis/>

Safari Extension Development

Non-Safari browser extensions are just raw HTML, JS, and CSS, and they can be loaded into the browser directly. Safari's browser extension implementation allows you to use a HTML/JS/CSS codebase, but running that codebase inside an Apple device requires a healthy amount of extra work. This section will cover the extra steps required to build a browser extension for Safari.

Note This section assumes you are either building a Safari extension from scratch, or porting a non-Safari HTML/JS/CSS extension codebase over to Safari.

Prerequisites

Safari browser extensions are developed and installed as apps and published to the App Store. Therefore, they must be built on Apple's app development infrastructure. You will need the following to develop and publish a Safari extension:

- A Mac computer with Xcode installed for development
- A developer certificate to sign your code (\$99/year)

Architecture

According to the Safari developer documentation:

A Safari web extension consists of three parts that operate independently in their own sandboxed environments:

- *A macOS or iOS app that can have a user interface*

- JavaScript code and web files that work in the browser

- A native app extension that mediates between the macOS or iOS app and the JavaScript code

This may sound confusing at first. To better understand how each of these parts works together, let's generate a new Xcode extension project, install it on an Apple device, and analyze each piece.

Creating an Extension Project

In Xcode, create a new project using the “Safari Extension App” template. For your first project, let's choose Swift as the language (Figures 15-12, 15-13, and 15-14).

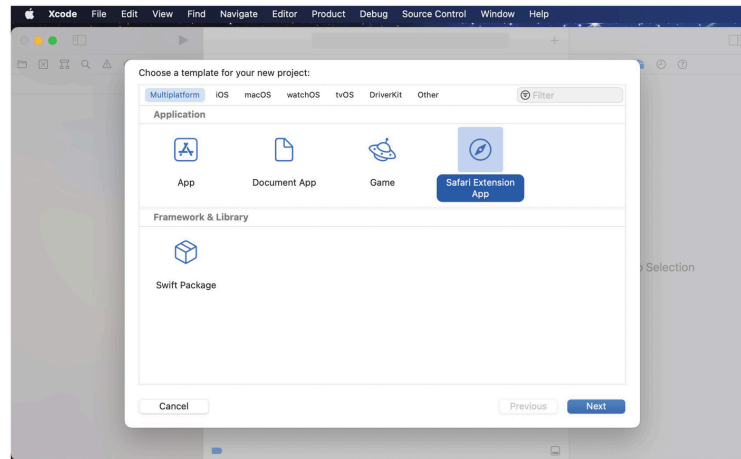


Figure 15-12 The Xcode Safari Extension App template

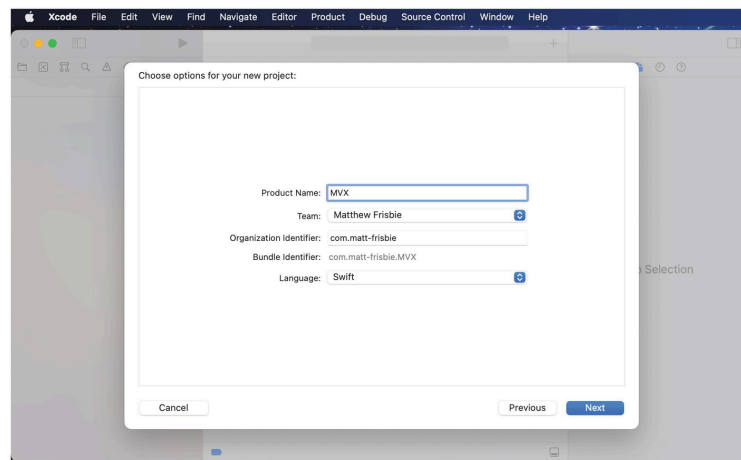


Figure 15-13 Setting options for the new project

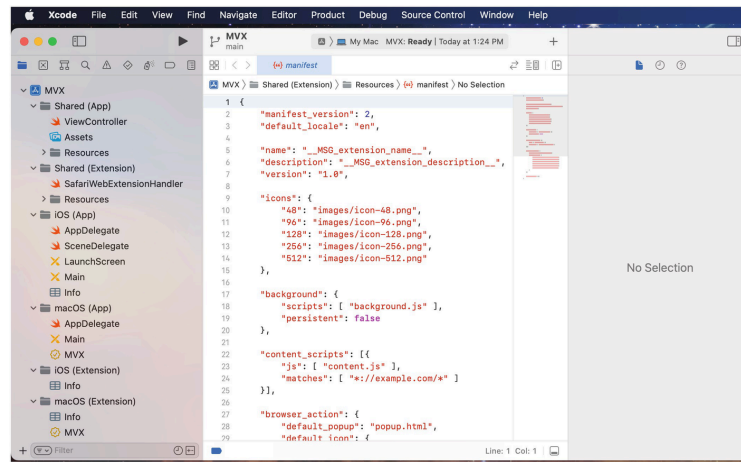


Figure 15-14 The new extension project's code

NoteThe Xcode template generates a manifest v2 extension, but Safari also supports manifest v3. We can ignore this difference for now.

Let's go through what each of these generated folders is:

- **iOS (App)** and **macOS (App)** are the formal apps that the OS will install. On iOS, this will show up as an app tile on the home screen. The generated version of these apps is just a simple web view.
- **Shared (App)** is the web view content. The generated version is just the default icon, some text, and a button.
- **iOS (Extension)** and **macOS (Extension)** are the native app extensions that mediate between the JS and the native app.
- **Shared (Extension)** is the formal extension code: manifest.json and JS/CSS/HTML, assets, locales, etc. This directory also includes an `ExtensionHandler` swift file that is used to handle messaging sent between the JS and the native app.

NoteXcode has separate build targets for iOS and macOS. This should make sense; although these apps share code, you are developing two entirely separate apps for two separate platforms.

Writing the App

The generated extension doesn't do much out of the box. Let's modify it to do the following:

- Use manifest v3
- Exchange message between the popup, background, and content script
- Show an options page
- Send messages to the native app

Update the XCode project files to match the following code:

manifest.json

```

{
  "manifest_version": 3,
  "default_locale": "en",
  "name": "__MSG_extension_name__",
  "description": "__MSG_extension_description__",
  "version": "1.0",

```

```

    "icons": {
      "48": "images/icon-48.png",
      "96": "images/icon-96.png",
      "128": "images/icon-128.png",
      "256": "images/icon-256.png",
      "512": "images/icon-512.png"
    },
    "options_ui": {
      "page": "options.html"
    },
    "background": {
      "service_worker": "background.js"
    },
    "content_scripts": [{
      "js": [ "content.js" ],
      "matches": [ "<all_urls>" ]
    }],
    "action": {
      "default_popup": "popup.html",
      "default_icon": {
        "16": "images/toolbar-icon-16.png",
        "19": "images/toolbar-icon-19.png",
        "32": "images/toolbar-icon-32.png",
        "38": "images/toolbar-icon-38.png",
        "48": "images/toolbar-icon-48.png",
        "72": "images/toolbar-icon-72.png"
      }
    },
    "permissions": [ "tabs", "nativeMessaging" ],
    "host_permissions": [ "<all_urls>" ]
  }

```

options.html

```

<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width">
  <meta charset="UTF-8">
</head>
<body>
  <h1>I'm the options page!</h1>
</body>
</html>

```

popup.html

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <link rel="stylesheet" href="popup.css">
  <script type="module" src="popup.js"></script>
</head>
<body>
  <h1>I'm the popup page!</h1>
  <button id="set-bg-color">Change background color</button>
  <button id="send-native-message">Send native message</button>
</body>

```

```
</html>
```

popup.js

```
document.querySelector("#set-bg-color").addEventListener("click", () => {
  const randomColor = "#" + Math.floor(Math.random()*(2 ** 24 - 1)).toString(16);
  chrome.tabs.query({
    active: true,
    currentWindow: true
  }, (tabs) => {
    chrome.tabs.sendMessage(
      tabs[0].id,
      { backgroundColor: randomColor },
      (response) => console.log(response)
    );
  });
});
document.querySelector("#send-native-message").addEventListener("click", () => {
  chrome.runtime.sendMessage(
    "com.matt-frisbie.MVX",
    {msg: "foobar"},
    (response) => console.log(response)
  );
});
```

content.js

```
console.log('Content script initialized!');
chrome.runtime.sendMessage({ greeting: "hello" }).then((response) => {
  console.log("Content script received response: ", response);
});
chrome.runtime.onMessage.addListener((request, sender, sendResponse) => {
  console.log("Content script received request: ", request);
  console.log("Sender: ", sender);
  if (request.backgroundColor) {
    document.body.style.backgroundColor = request.backgroundColor;
  }
});
```

background.js

```
chrome.runtime.onMessage.addListener((request, sender, sendResponse) => {
  console.log("Background received request: ", request);
  console.log("Sender: ", sender);
  if (request.greeting === "hello") {
    return sendResponse({ farewell: "goodbye" });
  }
});
```

Example 15-1.

SafariWebExtensionHandler.swift

```
import SafariServices
import os.log
let SFExtensionMessageKey = "message"
class SafariWebExtensionHandler: NSObject, NSExtensionRequestHandling {
```

```

func beginRequest(with context: NSExtensionContext) {
    let item = context.inputItems[0] as! NSExtensionItem
    let message = item.userInfo?[SFExtensionMessageKey]
    os_log(.info, "MVX App received: %{public}@", message as! CVarArg)
    let response = NSExtensionItem()
    response.userInfo = [ SFExtensionMessageKey: [ "App response to": message ] ]
    context.completeRequest(returningItems: [response], completionHandler: nil)
}
}

```

Most of this code should match what you’ve seen elsewhere in this book. For the most part, this is a conventional background/popup/options/content script setup for an extension. Bolded in the code are the portions needed to send messages from the extension JavaScript code to the native app Swift code. Some things to note here:

- The special permission `nativeMessaging` is needed to dispatch native messages.
- The `chrome.runtime.sendNativeMessage` is used to dispatch a native message.
- To target a native app, you must use the native app’s name. For Safari extensions, this is the App Bundle Identifier. In this example, the identifier is `com.matt-frisbie.MVX`

Testing on macOS

NoteIf you wish to develop Safari extensions without signing the app, you will need to enable Safari to load unsigned extensions. To do so, choose Safari ► Preferences, click Advanced, then select “Show Develop menu in menu bar.” Next, open the Develop menu and check “Allow unsigned extensions.”

Let’s first test this on desktop Safari. In Xcode, build the app for macOS (Figure 15-15).

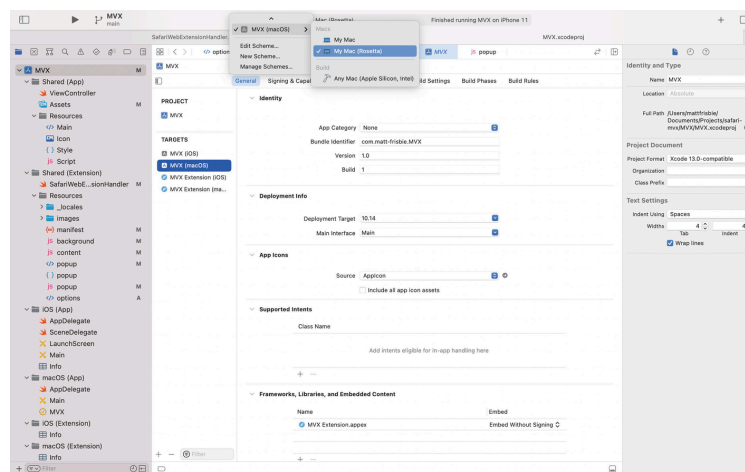


Figure 15-15 Building the extension for macOS

When the build finishes, you will see the extension app open (Figure 15-16).

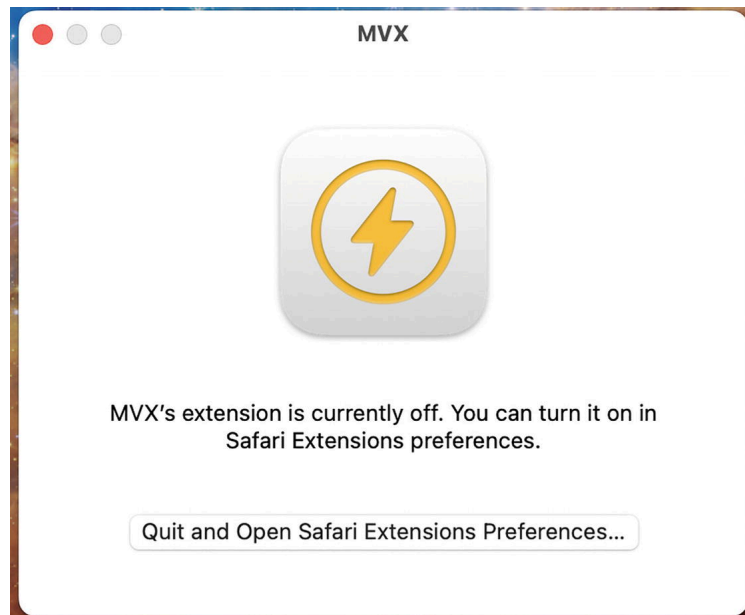


Figure 15-16 The macOS extension app

The view you are seeing is the native OS app displaying a web view. Xcode will have automatically installed the extension for Safari, so click this “Quit and Open...” button to open the browser’s preferences. Check the box next to “MVX” to enable the extension (Figure 15-17).

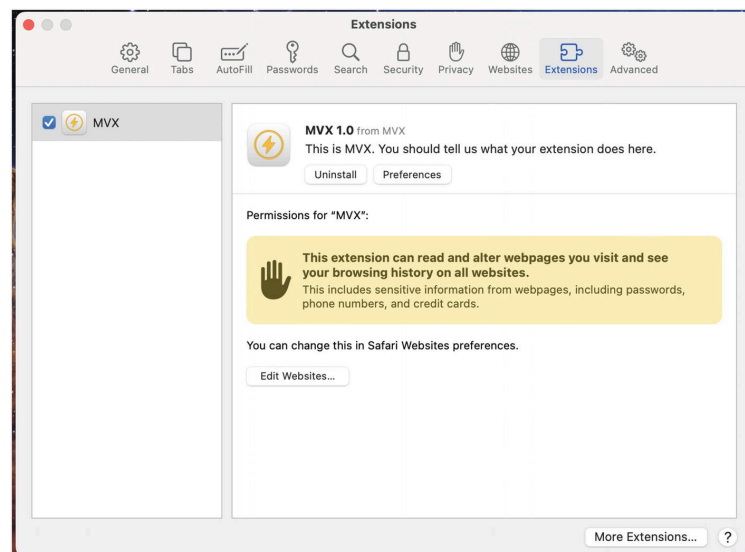


Figure 15-17 The enabled Safari extension

Note This app is requesting the `<all_urls>` host permission, so Safari will ask you to confirm the permission before enabling the extension.

Click the “Preferences” button, and your options page will open (Figure 15-18).

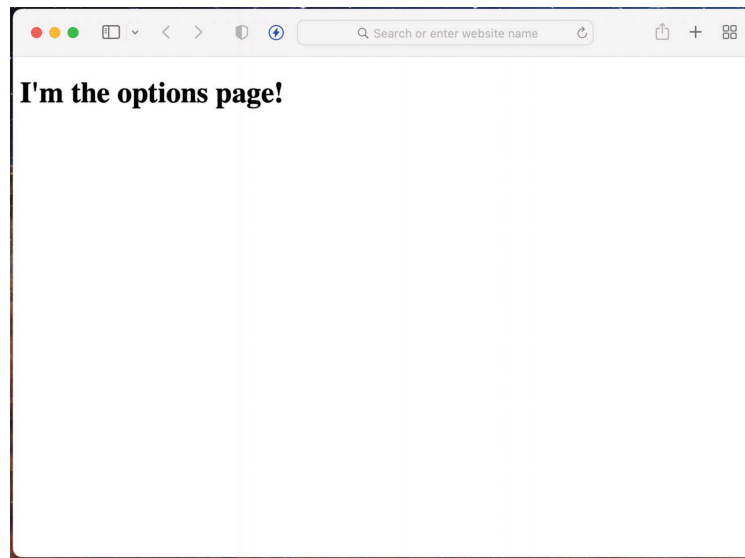


Figure 15-18 The options page

Next, navigate to a valid web page like wikipedia.org. Open the console to verify the content script is completing the message handshake with the background script (Figure 15-19).

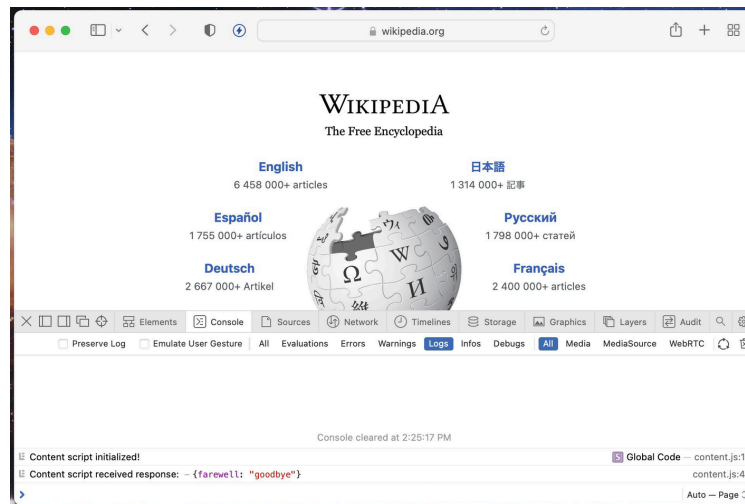


Figure 15-19 The content script logs

The default extension icon is the encircled lightning bolt, click it to open the popup page. Click the “Change background color” to set the background to a random color. You should see the content script log the message it received (Figure 15-20).

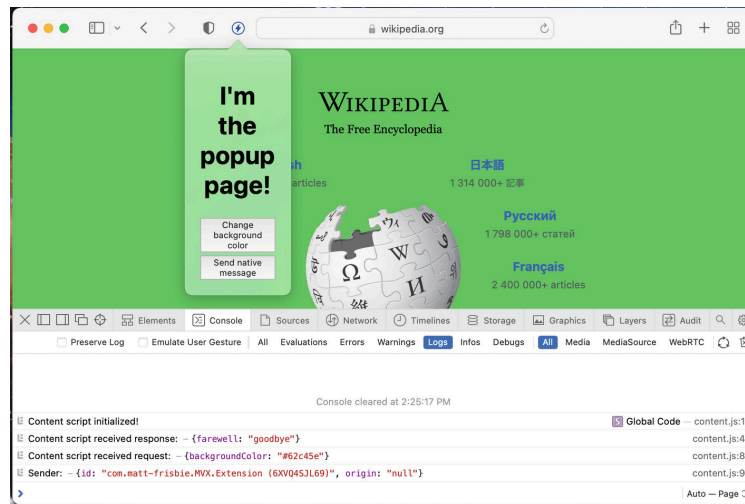


Figure 15-20 The popup page after a background color change

The second popup button will send a message to the background app. To check the logs for the background app, you will need to open the macOS Console app and filter for messages from the extension. Filter for messages from the MVX process, and for messages containing our dummy “foobar” message. The following screenshot shows the log output you should expect to see after clicking “Send native message” (Figure 15-21).

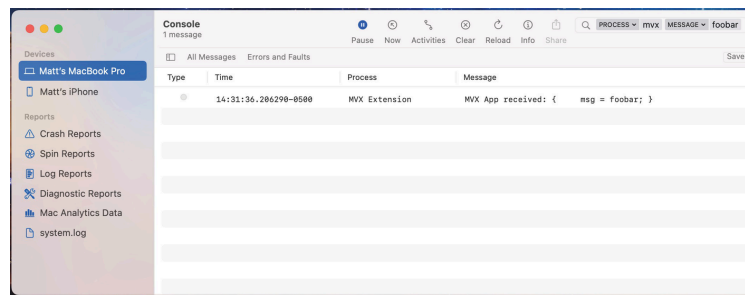


Figure 15-21 The os_log output from the macOS extension's native app

Testing on iOS

In Xcode, build the app for iOS, and use a simulator (Figure 15-22).

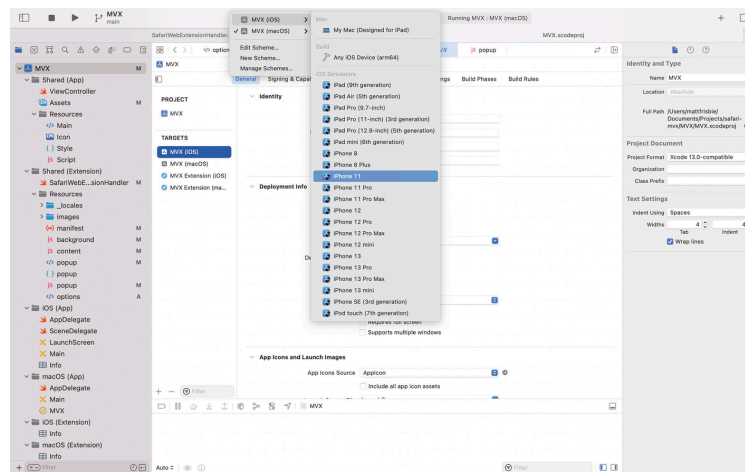


Figure 15-22 Build the extension for iOS and iPhone 11

The simulator will dump you right into the native app (Figure 15-23). The view you are seeing is the native OS app displaying a web view.

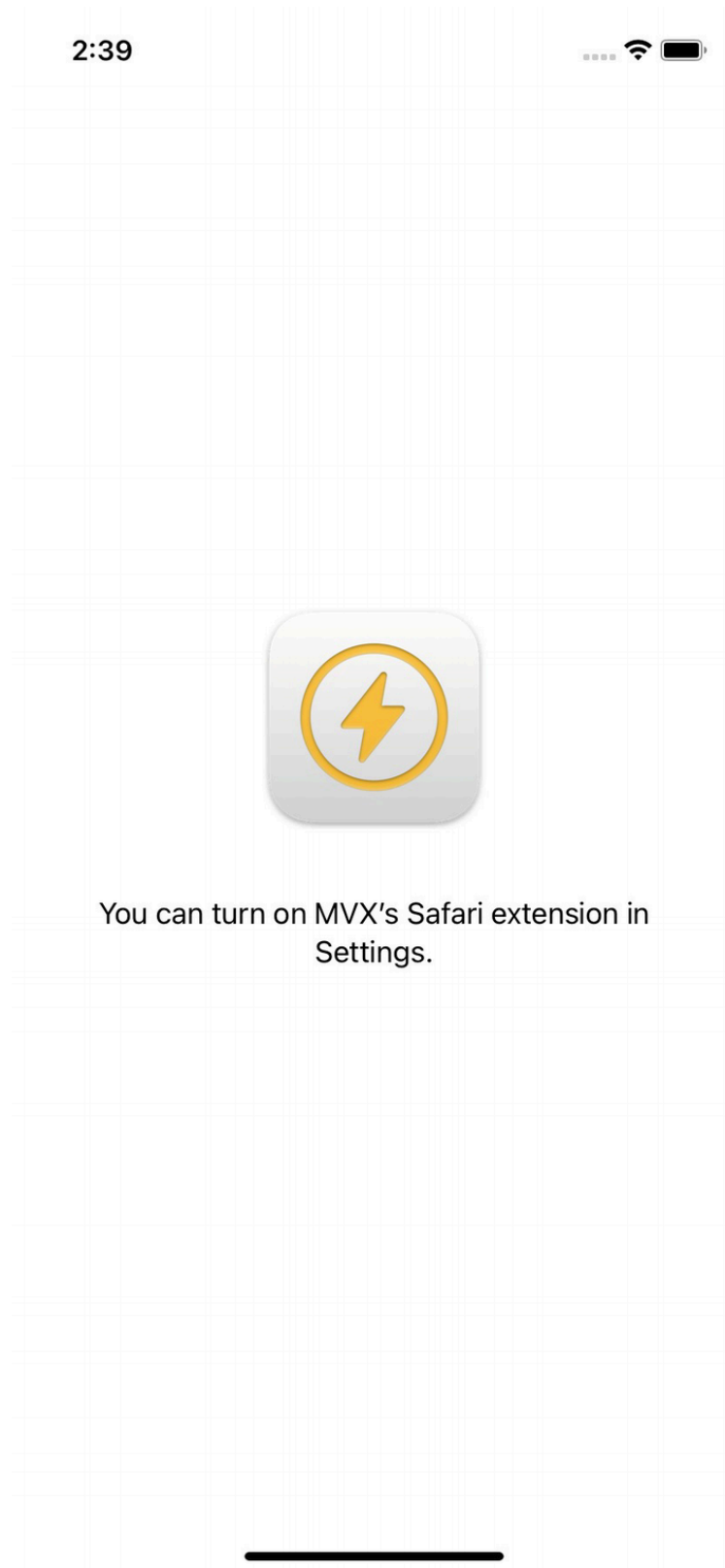


Figure 15-23 The native iOS app

Xcode will have automatically installed the extension. You can see this app by viewing the home screen (Figure [15-24](#)).

Figure 15-24 The extension app on the home screen

Safari extension apps can be managed by opening the Safari settings and selecting “Extensions” (Figures [15-25](#) and [15-26](#)).



Figure 15-25 iOS Safari settings showing an “Extensions” option

Figure 15-26 The MVX extension detail page on iOS

Clicking “Extension Settings” will open the options page (Figure [15-27](#)).

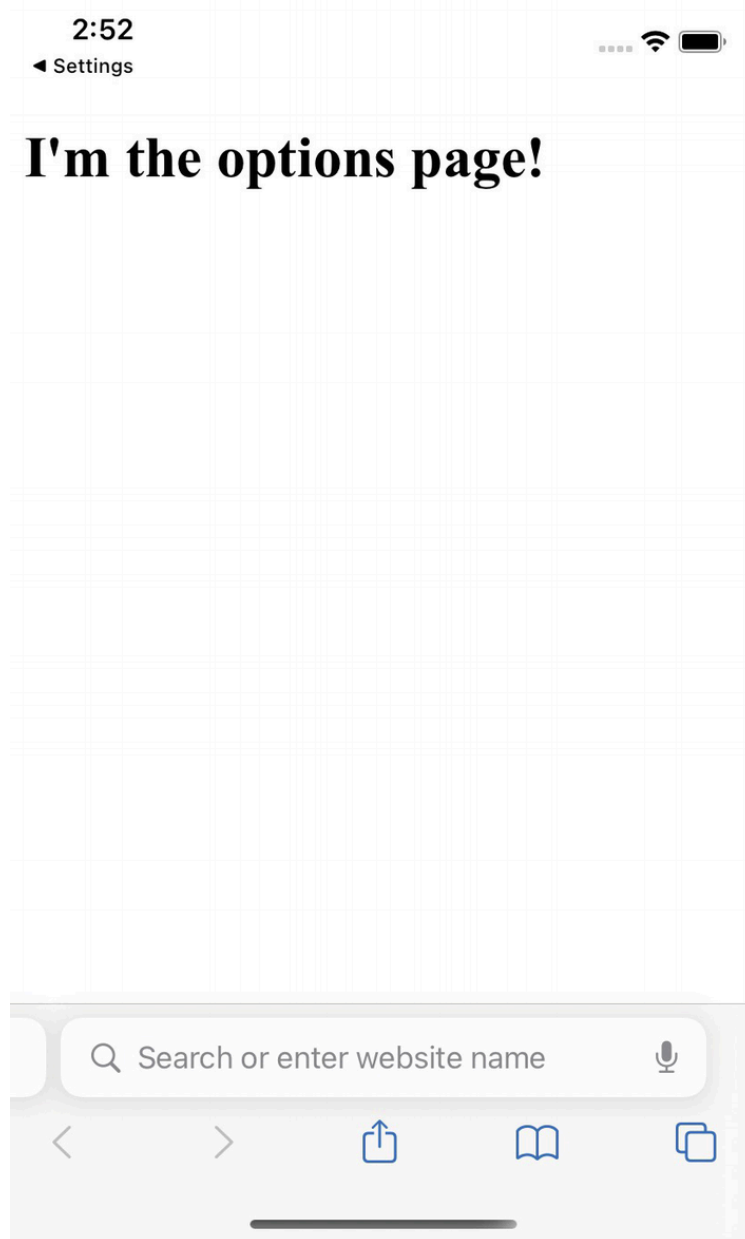


Figure 15-27 The options page on iOS Safari

Next, navigate to a valid web page like wikipedia.org. When iOS Safari has extensions installed, the URL bar will display a puzzle piece. Click this to reveal the menu, which should contain an MVX extension option (Figure [15-28](#)).



Figure 15-28 The iOS Safari extension menu options

This MVX option functions as the toolbar button, and selecting it will open the extension's popup page. Click the "Change background color" button to test that the content script is functioning (Figure [15-29](#)).



Figure 15-29 The popup page modifies the web page's background color

You can track the log messages created by clicking the “Send native message” button through the Console app in the same way as macOS Safari (Figure [15-30](#)).

Figure 15-30 The os_log output from the iOS extension's native app

Deploying to the App Store

You will need to deploy separate apps for iOS and macOS. The deployment process is no different than publishing regular iOS or macOS apps. At a high level, you'll need to perform the following steps:

1. Create a project archive
2. Build and sign the bundle
3. Validate the bundle
4. Push to the App Store to begin the approval process

An example macOS deploy flow is shown in Figures [15-31](#), [15-32](#), and [15-33](#).

Figure 15-31 The generated project archive used for publishing

Figure 15-32 The packaged project ready to be published

Figure 15-33 An example success screen when your project is successfully published

Converting an Existing Extension

Xcode has a command line utility built-in which will automatically convert an existing browser extension codebase into a Safari extension. The following command will generate an Xcode extension project including your existing files:

```
xcrun safari-web-extension-converter /path/to/extension
```

Tip More details on this CLI tool can be found here:

https://developer.apple.com/documentation/safariservices/safari_web_extensions/converting_a_web_e

Firefox Idiosyncrasies

Other than Safari, which is covered thoroughly in the previous extension, the Firefox add-on ecosystem has some notable differences that you should be aware of when developing for it.

Manifest Versions

At the time this book was written, Firefox is in a transitional period. The organization is attempting to soften the messy transition from manifest v2 to v3 by extending support for APIs like `webRequestBlocking` that developers wish to cling to. This state of things will likely not persist for far into the future, but for the time being, keep the following things in mind:

- Currently, Firefox is the only major browser that does not allow you to load manifest v3 extensions by default. It can be enabled, but its support is spotty.
- Currently, Firefox is currently the only major webstore that does not accept manifest v3 extensions.
- Firefox indicated that it will adopt manifest v3, but also preserve features that developers still need and use. What this ultimately will look like is unclear.

Additionally, when manifest v3 is eventually supported, you will need an add-on ID in your manifest. The MDN documentation includes the following:

All Manifest V3 extensions need an add-on ID in their manifest.json when submitted to AMO. Contrary to Manifest V2 extensions, AMO will not accept Manifest V3 extensions without an ID and it will not automatically embed this ID in the signed packaged extension.

The add-on ID is declared under

`browser_specific_settings.gecko.id`. For example:

manifest.json

```
{
  ...
  "browser_specific_settings": {
    "gecko": {
      "id": "browserextensionexplorer@buildingbrowserextensions.com"
    }
  },
  ...
}
```

NoteRead the Mozilla announcement here:

<https://blog.mozilla.org/addons/2022/05/18/manifest-v3-in-firefox-recap-next-steps/>.

Sidebars

Firefox sidebars are extension-controlled containers that are defined similarly to popups. They are declared in the manifest under the `sidebar_action` key and appear alongside the active web page on either side (Figure [15-34](#)).

Figure 15-34 A Mozilla Firefox sidebar

API Additions

Firefox is unlike other browsers in that it adds a significant number of APIs to the extension API namespace:

- `captivePortal`
- `contextualIdentities`
- `dns`
- `find`
- `menus`
- `pckssl`
- `sidebarAction`
- `theme`
- `userScripts`

TipFor a summary of these API differences, refer to this link:

https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Firefox_differentiators.

Summary

In this chapter, we covered all the complicated aspects of creating an extension that supports multiple browsers. We discussed how an extension built on the Chrome Web Store can support a vast majority of desktop browsers that are based off the Chromium project. We also discussed all the different places you can deploy a browser extension as well as different ways extensions can be used on mobile.

This chapter also covered various browser idiosyncrasies. It described in detail how to develop and publish extensions for Safari on iOS and macOS, as well as all the unique aspects of developing for Firefox.

In the next chapter, we will cover all the different tools, platforms, and frameworks that you can use to make extension development and deployment easier.